Failure-Aware Self-Triage in LLM Agents

14-MONTH MASTER RESEARCH & EXECUTION PLAN

Rohit & Bhawika | Sister Nivedita University, New Town, Kolkata, West Bengal

BCA Year 2 | May 2026 – August 2027

RESEARCH QUESTION:

"Can an AI agent learn to recognise what kind of mistake it just made, and use that to decide the smartest next move — stop, retry, self-fix, or ask for help?"

TARGET: NeurIPS/ICML 2027 Workshop

GOAL: Google Internship + Full-time

Duration	Start	University	Hardware A	Hardware B	Target Venue
14 Months	May 2026	Sister Nivedita Univ, Kolkata	MacBook M3 Max 128GB/4TB	RTX 4090 Ti 64GB/4TB	NeurIPS/ICML 2027

Primary Research Question	*Can an AI agent learn to recognise what kind of mistake it just made and use that to decide the smartest next move — stop, retry, self-fix, or ask for help?*

#	Sub-Question	Why It Matters	Hardware Used
SQ1	Do AI agents become better when they remember what went wrong in past tasks?	Addresses memory & learning in agents — directly relevant to Google DeepMind's agent memory research	MacBook M3 Max (fast iteration)
SQ2	Can a small local model (7B) act as effectively as GPT-4 as an agent if you give it better tools?	Proves that RTX 4090 Ti + smart design beats expensive API calls — NVIDIA's core narrative	Windows PC / RTX 4090 Ti (local LLM)

0 PHASE MAP — 14 Months at a Glance

Phase	Months	Period	Theme	Rohit Focus	Bhawika Focus	Key Deliverable
PRE	0	May–Jun 2026	Runway	Papers + env setup	Python basics CS50P	GitHub org + 15 tasks
P1	1–2	Jul–Aug 2026	Build	Baseline + Triage agent	150 tasks + baseline run	Working dual-agent system
P2	3–4	Sep–Oct 2026	Experiment	Ablations + debug	Annotate + visualise	Results JSONL + figures
P3	5–6	Nov–Dec 2026	Write	Method + intro	Results + setup sections	Paper v1 draft in Overleaf
P4	7–8	Jan–Feb 2027	Polish	Review + arXiv	Final figures + proofs	arXiv submission
P5	9–10	Mar–Apr 2027	Apply	Internship apps	Google STEP / GirlScript	First interview callback
P6	11–12	May–Jun 2027	Present	Conference/poster	Repo + project page	Public GitHub + talk
EXT	13–14	Jul–Aug 2027	Extend	Memory module (SQ1)	SQ2 small-vs-large eval	Extended paper v2

1 WHAT YOU ARE BUILDING — System Overview

You are building two LLM agent systems and running a controlled experiment on 150 tasks to prove that failure-aware triage is smarter than blind retrying. The project is framed around three hardware-optimised environments across your two machines.

	MacBook M3 Max	Windows / RTX 4090 Ti
RAM	128 GB unified	64 GB DDR5
Storage	4 TB SSD	4 TB NVMe
CPU	M3 Max 16-core	Ryzen 9 7950X (16C/32T)
GPU	Apple Neural Engine	RTX 4090 Ti — MAIN LLM GPU
Role	Paper writing, Overleaf, light Python, API calls	LLaMA 3.1 8B inference, all experiments, benchmarks
OS	macOS Sequoia	Windows 11 + WSL2 Ubuntu
Key tools	VS Code, Overleaf, Notion	Ollama, LangGraph, PyTorch, CUDA
Rohit uses for	Writing code logic, designing prompts	Running full benchmark (150 tasks × 2 agents)
Bhawika uses for	Task JSON creation, matplotlib figs	Running analysis scripts

The Two Systems You Will Build and Compare

	Baseline Agent (blind-retry)	Triage Agent (your contribution)
Core idea	When a tool fails → increment counter → if > 3 retries → FAIL	When a tool fails → classify failure type → choose smart recovery → act
LLM	LLaMA 3.1 8B via Ollama on RTX 4090 Ti	Same model + extra triage prompt call
Framework	LangGraph StateGraph (Python)	LangGraph + failure_triage.py module
Tools	5 tools: web_search, file_read, calculate, code_exec, api_call	Identical 5 tools
Failure handling	Dumb counter. No reasoning.	6 failure types × 5 recovery strategies = smart decision
State tracked	steps_taken, tool_calls, failures, retry_counts	All baseline state + triage_decisions list + classification_correct flag
File name	baseline_agent.py	triage_agent.py + failure_triage.py
Built by	Rohit (July 2026)	Rohit (August 2026)

The 6 Failure Types Your Agent Must Recognise

#	Type ID	What It Means	Real Example	Best Recovery
1	tool_unavailable	External API is down, endpoint 404, service unreachable	DuckDuckGo search returns 503 Service Unavailable	retry_same_step (wait 2s)
2	malformed_input	Request formatted wrong — bad JSON, missing required field, wrong type	API expects {"q": "..."} but agent sent plain string	fix_and_retry
3	execution_error	Tool crashed mid-run, hit timeout, out of memory, exception thrown	Python code_exec node hits 30s timeout on infinite loop	retry_same_step or fix_and_retry
4	logic_error	Tool ran successfully but result is wrong — agent chose wrong action	Agent searched 'Bitcoin 2019 price' instead of current price	backtrack_and_replan

#	Type ID	What It Means	Real Example	Best Recovery
5	irreversible	Action succeeded but CANNOT be undone — data deleted, purchase made	Agent deleted bitcoin_price.txt when asked to overwrite it	escalate_to_human or abort
6	ambiguous_task	Original task description is unclear, contradictory, or underspecified	'Find the best result' — best by what metric?	escalate_to_human

The 5 Recovery Strategies

Recovery Action	Trigger Condition	What the Agent Does	Expected Outcome
retry_same_step	tool_unavailable, execution_error	Wait 2 seconds. Retry with identical inputs.	Works when failure was transient (network blip, timeout)
fix_and_retry	malformed_input, logic_error	Ask LLM to revise the tool input or approach. Then retry.	Works when the plan was right but the call was wrong
backtrack_and_replan	Deep logic_error (>1 wrong step)	Pop last N steps from state. Return to planning node. Choose different path.	Works when the agent went down the wrong branch
escalate_to_human	irreversible, ambiguous_task	Log 'NEEDS_HUMAN'. Set task_success=False. Pause.	Prevents catastrophic or irreversible errors
abort_task	irreversible at critical step, repeated failures	Set is_complete=True, task_success=False. Do not retry.	Last resort — better to fail clean than cause damage

2 PRE-YEAR PHASE — May 16 to June 30, 2026 (6 Weeks)

Your last exam is this Friday (May 16). Year 2 starts July 7. This is your 6-week runway. No pressure to write production code yet — but use this window to read papers, set up tools, and build the shared foundation so July hits the ground running.

Week	Dates	Rohit — Learning & Setup	Bhawika — Learning & Setup	Shared
W1	May 19–25	Read ReAct paper (arxiv 2210.03629). Watch 'LangGraph crash course' on YouTube. Skim LangChain docs.	Enrol CS50P (cs50.harvard.edu/python). Watch Week 0–2. Variables, loops, if-else. Write first 5 programs.	Create GitHub org: failure-triage-research. Create repo: agent-triage. Add both as collaborators.
W2	May 26–Jun 1	Read AgentBench paper (2308.03688, 45 min). Install Ollama on Windows PC. Run: ollama pull llama3.1:8b. Verify it loads.	CS50P Weeks 3–4. Functions, exceptions, libraries. Do all problem sets. Ask Rohit to explain any stuck concept.	Rohit helps Bhawika set up Python 3.11 + VS Code on her machine. Both learn git add/commit/push together.
W3	Jun 2–8	Read AgentRx carefully (2602.02475, 90 min). Write 1-page note: how their 9-category taxonomy maps to your 6 types.	CS50P Weeks 5–6. File I/O, JSON. Practice: read a JSON file, print each field. This is the skill you need for tasks.	30-min call. Rohit explains AgentRx to Bhawika simply. Together draft the task JSON template format.
W4	Jun 9–15	Read Chain-of-Thought (2201.11903, 45 min) + MAST skim (2503.13657, 30 min). Draft triage prompt template.	CS50P Weeks 7–8. Pandas basics. Load a CSV, filter rows, print summary. Learn matplotlib: plot a bar chart.	Together: finalise the task JSON schema. Both add it to a shared Notion doc.
W5	Jun 16–22	Read WebArena (2307.13854, 60 min). Set up dev environment: pip install langgraph langchain pytest pandas matplotlib seaborn.	Write first 15 task JSON files manually (5 web_search, 5 file_ops, 5 calculation). Validate with python -c 'import json; json.load(open("task.json"))'	Push 15 tasks to GitHub. Write README.md together. Rohit reviews Bhawika's tasks and gives feedback.
W6	Jun 23–30	Write skeleton of baseline_agent.py — define all TypedDicts, all function signatures, graph structure. No logic yet.	Write 15 more tasks (30 total). Begin understanding tools.py design by reading Rohit's skeleton.	Tag repo v0.1-skeleton. Screen-share review session. Celebrate: you have a GitHub repo, a skeleton, and 30 tasks.

All 6 Papers — Reading Guide with Notes

#	Paper & Year	ArXiv	Time	Your Role in Pipeline	Key Fact to Extract
1	ReAct: Reasoning+Acting Yao et al., ICLR 2023	2210.03629	60 min	Foundation — your baseline_agent IS a ReAct agent	The think-act-observe loop. Draw this on paper. Your triage node sits between observe and the next act.
2	AgentBench Liu et al., ICLR 2024	2308.03688	45 min	Benchmark methodology — copy their task design	How they define 'success' across 8 environments. Directly shapes your success_criteria JSON field.
3	WebArena Zhou et al., ICLR 2024	2307.13854	60 min	Gold standard eval — your paper cites this as prior art	How they handle partial credit. What 'task completion rate' means formally.

#	Paper & Year	ArXiv	Time	Your Role in Pipeline	Key Fact to Extract
4	AgentRx Barke et al., Feb 2026	2602.02475	90 min	CLOSEST COMPETITOR — differentiate on recovery, not diagnosis	Their 9-category taxonomy. Map to your 6. Write: 'AgentRx diagnoses; we recover.'
5	MAST Cemri et al., 2025	2503.13657	30 min	Annotation methodology — use their inter-annotator agreement method	How to validate human annotations of failure labels. Bhawika uses this when annotating 150 tasks.
6	Chain-of-Thought Wei et al., NeurIPS 2022	2201.11903	45 min	Prompt design — your triage prompt IS a CoT prompt	Step 1 → Step 2 → JSON structure. Your classify_failure() prompt uses this exact pattern.

Environment Setup Checklist

■ ROHIT

◆ ROHIT SETUP — Windows PC (RTX 4090 Ti)

- Install Ollama: curl https://ollama.ai/install.sh | sh
- ollama pull llama3.1:8b (takes ~5GB download)
- Verify: curl http://localhost:11434/api/tags
- pip install langgraph langchain langchain-community
- pip install langchain-ollama pytest pandas matplotlib seaborn
- pip install requests duckduckgo-search
- VS Code + Python extension + GitLens + GitHub Copilot
- WSL2 Ubuntu 22.04 for Linux tools
- MacBook M3 Max: Xcode CLT, Homebrew, git, VS Code
- Overleaf account (overleaf.com) — free tier is fine

■ BHAWIKA

◆ BHAWIKA SETUP — Her Machine

- Python 3.11 from python.org
- VS Code from code.visualstudio.com
- Git from git-scm.com
- CS50P enrolment: cs50.harvard.edu/python (FREE)
- GitHub account — join shared org
- Notion account — shared project board
- Learn: git clone, add, commit, push, pull
- Install: pip install pandas matplotlib seaborn
- Practice: run print('Hello, Research!') successfully
- Buy: dedicated A5 notebook for research notes

3 PHASE 1 — FOUNDATIONS & CORE BUILD | July – August 2026

Year 2 begins July 7. These two months are the most critical of the entire 14-month plan. By August 31 you must have: a working baseline agent, a working triage agent, all 150 tasks, and the first batch of results. Everything else in the plan depends on this.

■ *Rohit's birthday: August 24 — keep August 20–26 lighter on deadlines. Celebrate the milestone: by your birthday you should have the triage agent running!*

July 2026 — Baseline Agent

Week	Dates	Rohit Builds / Learns	Bhawika Builds / Learns	Shared Checkpoint
W1	Jul 7–13	LEARN: Re-read LangGraph StateGraph docs. Understand: nodes, edges, conditional_edges, State TypedDict. Write 3 hello-world LangGraph programs.	LEARN: Reread your 30 task JSONs. Understand every field. Write 10 more tasks (calc + code categories).	Both: read the AgentBench paper together (Rohit explains). Understand why 150 tasks across 6 categories.
W2	Jul 14–20	BUILD: tools.py — implement all 5 tool functions. Each must: accept typed input, return ToolResult TypedDict, handle exceptions gracefully.	BUILD: Write 20 more tasks (api_call + multi_step categories). Total: 60 tasks done.	Rohit demos tools.py running on MacBook. Bhawika tests manually: call each tool with example input.
W3	Jul 21–27	BUILD: baseline_agent.py — implement plan_action(), execute_tool(), should_retry(). Wire StateGraph. Run on 5 tasks manually.	Write 30 more tasks (multi_step + remaining categories). Total: 90 tasks done. Review all for quality.	Integration test: run baseline agent on Bhawika's tasks. Log output. Both review failures together.
W4	Jul 28–31	DEBUG: Fix all errors from integration test. Add logging to every node. Write docstrings. Push tag v0.2-baseline.	Write final 60 tasks. Total: 150 tasks. Run validation script on all 150 JSONs.	Final July push: all 150 tasks in tasks/ folder. baseline_agent.py runs on 10 tasks without crashing.

August 2026 — Triage Agent + First Experiments

Week	Dates	Rohit Builds	Bhawika Builds	Deliverable
W1	Aug 3–9	Build failure_triage.py: FailureType enum, RecoveryAction enum, TriageDecision TypedDict, classify_failure() function with full prompt.	Run baseline agent on first 30 tasks. Log all results to baseline_results_part1.jsonl	failure_triage.py unit tested: 10 mock failures → correct classification ≥7/10
W2	Aug 10–16	Build triage_agent.py: extend baseline with triage node. Wire: execute → triage → decision. Test on 10 tasks.	Run baseline on next 60 tasks. Manually inspect 5 failure logs — understand what goes wrong.	triage_agent.py runs 10 tasks without crash. triage_decisions logged in state.
W3	Aug 17–23	Build benchmark.py: BenchmarkRun class, --agent flag, --tasks flag, --output flag. Runs all 150 tasks sequentially.	Complete baseline run on all 150 tasks. baseline_results.jsonl: 150 lines, all valid JSON.	One-command baseline run: python benchmark.py --agent baseline --tasks tasks/ --output baseline_results.jsonl

Week	Dates	Rohit Builds	Bhawika Builds	Deliverable
W4	Aug 24–31	BIRTHDAY WEEK (Aug 24). Debug any triage failures. Write 20 pytest unit tests. Tag v0.3-triage.	Learn to parse JSONL with pandas. Count: how many tasks succeeded? Write first summary table.	Both systems run. v0.3-triage tagged. Birthday milestone: THE CORE SYSTEM IS BUILT.

All Files You Must Deliver by Aug 31

File	What It Contains	Who	Deadline	Pass Criteria
tools.py	5 tool functions: web_search (DuckDuckGo API), file_read, calculate (safe eval), code_exec (subprocess with timeout), api_call (requests.get/post). Each returns ToolResult TypedDict.	Rohit	Jul 20	All 5 tools return correct ToolResult. Exception handling for all error cases.
baseline_agent.py	Full LangGraph StateGraph: plan_action, execute_tool, should_retry nodes. Blind retry logic. Logs all steps.	Rohit	Jul 28	Runs 10 tasks end-to-end. Writes results dict with all required fields.
failure_triage.py	FailureType + RecoveryAction enums. TriageDecision TypedDict. classify_failure(). apply_recovery(). Full triage prompt.	Rohit	Aug 9	Unit test: 10 mock failures → ≥7 correct classifications.
triage_agent.py	Extended baseline with triage node. Uses failure_triage.py on every failure. Stores triage_decisions in state.	Rohit	Aug 16	Runs 10 tasks. triage_decisions list populated after each failure.
benchmark.py	BenchmarkRun class. Argparse CLI. Runs any agent on any task file. Writes JSONL incrementally.	Rohit	Aug 23	One command runs 150 tasks. Output is valid JSONL.
tasks/ (150 files)	25 tasks × 6 categories. Every JSON has: task_id, category, description, expected_tools, success_criteria, expected_failure, recovery_hint, difficulty, created_by.	Bhawika	Aug 15	All 150 load without JSON error. Validated by: python validate_tasks.py tasks/
baseline_results.jsonl	150-line JSONL. Each line: task_id, category, agent_type, task_success, num_steps, num_failures, num_retries, elapsed_time, tokens_used.	Bhawika	Aug 31	150 valid lines. Both success and failure cases present.
README.md	Project overview, setup instructions, how to run each script, example commands.	Both	Aug 31	A new reader can set up and run the project in under 30 minutes.

Task JSON — Exact Template (Bhawika: copy this for every file)

```
{
  "task_id": "web_001",
  "category": "web_search",
  "description": "Search for the current price of Bitcoin in USD and save it to bitcoin_price.txt.",
  "expected_tools": ["web_search", "file_write"],
  "success_criteria": "File bitcoin_price.txt exists AND contains a USD price from the last 24 hours",
  "expected_failure": "tool_unavailable",
  "recovery_hint": "retry_same_step",
  "difficulty": "easy",
  "created_by": "Bhawika",
  "notes": "DuckDuckGo may rate-limit. Agent should wait 2s before retry."
}
```

150 Task Categories — 25 Each

Category	task_id Range	Task Pattern	Primary Failure Injected	Example Task
web_search	web_001–025	Search for X. Find current Y. Look up Z on the web.	tool_unavailable (API down) logic_error (wrong query)	Find the current price of Bitcoin in USD and save to bitcoin_price.txt
file_operations	file_001–025	Read file X. Write content Y to Z. Copy A to B. Append C to D.	execution_error (file locked) irreversible (deleted)	Read data.csv and count the number of rows. Save the count to row_count.txt
calculation	calc_001–025	Compute X. Convert Y units. Find Z percentage. Solve equation.	malformed_input (bad eval) logic_error (wrong formula)	Calculate compound interest on 10000 at 8% for 5 years and write the result
code_execution	code_001–025	Run this Python snippet. Fix this broken code. Execute script with input X.	execution_error (crash/timeout) irreversible (side effects)	Execute the following Python code and return its output: [code with subtle bug]
api_calls	api_001–025	Call this REST endpoint. Fetch JSON from URL. POST data to API.	tool_unavailable (403/404) malformed_input (wrong headers)	Fetch the top post from r/MachineLearning using Reddit's public API and save the title
multi_step	multi_001–025	Search THEN compute THEN save. Chains 2–3 tools in sequence.	ambiguous_task (unclear chain) logic_error (wrong step order)	Find today's USD to INR exchange rate, multiply by 5000, and save the result to conversion.txt

4 PHASE 2 — EXPERIMENTS & ANALYSIS | Sep – Oct 2026

You have a working system. Now you collect the evidence that makes your paper publishable. Rohit runs ablations; Bhawika annotates ground-truth labels and builds all visualisation figures.

Month	Week	Rohit	Bhawika	Output File
Sep	W1 (Sep 1–7)	Run triage agent on all 150 tasks on RTX 4090 Ti. Monitor GPU usage. Log all triage_decisions.	Manually annotate ground-truth failure type for first 30 tasks from baseline_results.jsonl	triage_results.jsonl (150 lines)
Sep	W2 (Sep 8–14)	Compare triage agent's classifications vs Bhawika's ground-truth. Compute first accuracy number.	Continue annotating: 60 tasks done. Use AgentRx and MAST taxonomy as reference.	annotation_gt_60.csv
Sep	W3 (Sep 15–21)	Write pytest test suite: 20 unit tests for classify_failure(), 10 integration tests for triage_agent.	Finish all 150 annotations. Compute inter-annotator agreement (Rohit also labels 20 tasks; compare).	annotation_ground_truth.csv (150 rows FINAL)
Sep	W4 (Sep 22–30)	Fix any bugs revealed by test suite. Re-run any tasks that crashed.	Build analysis.py skeleton: load JSONL files, compute basic stats (mean success rate, mean retries).	tests/ folder. analysis.py skeleton.
Oct	W1 (Oct 1–10)	Ablation 1: Remove classification from triage — always RETRY regardless of type. Measure degradation.	Generate Fig 1 (overall success rate bar chart) and Fig 2 (retry count box plot).	ablation_always_retry.jsonl. fig1_success.png, fig2_retries.png
Oct	W2 (Oct 11–17)	Ablation 2: Replace LLaMA triage with rule-based heuristics (no LLM). Does the LLM add value?	Generate Fig 3 (confusion matrix: predicted vs true failure type) using sklearn + seaborn heatmap.	ablation_rule_based.jsonl. fig3_confusion.png
Oct	W3 (Oct 18–24)	Ablation 3 (SQ2): Use GPT-4o-mini via API for triage only. Compare accuracy vs LLaMA 3.1 8B.	Generate Fig 4 (per-category breakdown grouped bar chart) and Fig 5 (token efficiency violin plot).	ablation_gpt4omini_triage.jsonl. fig4_per_cat.png, fig5_tokens.png
Oct	W4 (Oct 25–31)	Write comparison.csv: side-by-side metrics for all 4 conditions (baseline, triage, ablation1, ablation2).	Generate Fig 6 (recovery action distribution pie chart). Write figure captions for all 6 figures.	comparison.csv. fig6_recovery.png. figure_captions.md

All 6 Figures — Exact Specs for Bhawika

Fig #	Title	Chart Type	X-axis	Y-axis	Colours	File
1	Overall Task Success Rate	Grouped bar chart	Agent type (Baseline / Triage / Ablations)	Success rate 0–1.0	Baseline=DGRAY, Triage=ACCENT, Ablations=MGRAY	fig1_success_rate.png
2	Retry Count Distribution	Box plot	Agent type	Number of retries per task	Same as Fig 1	fig2_retry_dist.png
3	Failure Classification Accuracy	Heatmap (confusion matrix)	True failure type (6 rows)	Predicted failure type (6 cols)	White=0, Dark blue=high count	fig3_confusion_matrix.png

Fig #	Title	Chart Type	X-axis	Y-axis	Colours	File
4	Success Rate by Task Category	Grouped bar per category	6 categories	Success rate	Baseline vs Triage side-by-side	fig4_per_category.png
5	Token Usage per Task	Violin plot	Agent type	Tokens used per task	Baseline=orange, Triage=teal	fig5_token_efficiency.png
6	Recovery Action Distribution	Pie chart (triage only)	n/a	Percentage of each recovery chosen	6 distinct colours per action	fig6_recovery_actions.png

Key Metrics to Compute (analysis.py must output all of these)

Metric	Formula	What a Good Result Looks Like
Task Success Rate	tasks_succeeded / total_tasks	Triage > Baseline by ≥10 percentage points
Unnecessary Retry Rate	retries_on_non_transient_failures / total_retries	Triage much lower — it doesn't retry irreversible failures
Triage Classification Accuracy	correctly_classified_failures / total_failures	Target ≥65% — this is your main accuracy metric
Recovery Action Appropriateness	recoveries_that_led_to_success / total_recoveries	Target ≥50% — chosen strategy actually worked
Token Efficiency	mean tokens per completed task	Triage may use slightly more tokens — but succeeds more
Mean Retries per Task	sum(num_retries) / 150	Triage should be lower even with extra triage calls
Per-Category Success Gap	triage_success_rate[cat] - baseline_success_rate[cat] for each of 6 cats	Find which categories benefit most from triage
Ablation Delta	triage_metric - ablation_metric	Proves each component contributes

5 PHASE 3 — PAPER WRITING | Nov – Dec 2026

You have results. Now write the 8-page paper. Use Overleaf (overleaf.com). Template: NeurIPS 2025 or ACL 2025 format — search 'NeurIPS 2025 LaTeX template' on Overleaf. Write section by section. Never write the abstract first — write it last.

Section	Pages	Word Target	Who Writes	Deadline	What to Write
Abstract	0.25	150 words	Both	Dec 5	Write LAST. 4 sentences: (1) Problem agents fail blindly, (2) Your solution — triage module, (3) Key result — X% better success Y% fewer retries, (4) Impact. MAX 150 words.
1. Introduction	1.0	~500 words	Rohit	Dec 15	Para 1: Hook — real agent failures in 2025-26 (cite NYC chatbot, Replit incidents). Para 2: Why blind retry fails. Para 3: Your solution. Para 4: Contributions — 3 bullet points: (a) 6-type taxonomy, (b) triage module, (c) 150-task benchmark. Last sentence: 'Code and benchmark released at [GitHub URL].'
2. Related Work	0.75	~350 words	Rohit	Dec 22	3 short subsections: (a) Agent failure taxonomies — cite AgentRx, MAST. 'They classify; we recover.' (b) Agent evaluation — cite AgentBench, WebArena. (c) Error handling in SE — brief nod to exception handling tradition as inspiration.
3. Method	1.5	~700 words	Rohit	Jan 5	3 subsections: (a) Baseline agent — LangGraph diagram, 5 tools, state definition. (b) Failure taxonomy — 6-type table with formal definitions. (c) Triage module — full decision flowchart, classify_failure() prompt in a figure box, apply_recovery() decision table.
4. Experimental Setup	1.0	~450 words	Bhawika	Jan 10	3 subsections: (a) Benchmark — 150 tasks, 6 categories, how tasks were created, validation process. (b) Metrics — define all 8 metrics formally. (c) Hardware + reproducibility — RTX 4090 Ti, LLaMA 3.1 8B, Ollama version, seed, one-command reproduce instructions.
5. Results	1.5	~700 words	Both	Jan 20	Lead with main result table (Table 1: all 4 conditions × all metrics). Then sub-sections for: Fig 1 (success rate), Fig 3 (confusion matrix analysis), Fig 4 (per-category — which categories benefit most?), ablation study results. Be precise: 'Triage achieves 73.3% success vs baseline 61.3% ($p<0.05$ McNemar test).'
6. Discussion	0.5	~250 words	Rohit	Jan 25	3 paragraphs: (a) What worked — which failure types did triage handle best? (b) What did not — false escalations, over-confident classifications. (c) Limitations — local 8B model only, single-agent scope, synthetic task distribution.
7. Conclusion	0.25	~100 words	Both	Jan 28	3 sentences: (1) We showed X. (2) Our open benchmark enables future work. (3) Future: memory integration (SQ1), multi-agent extension.
References	0.25	~6-8 refs	Both	Jan 28	All 6 papers you read + any extra cited in intro. Use BibTeX. Download .bib entries from arxiv.org.

November Writing Schedule — Week by Week

Week	Rohit	Bhawika
Nov W1 (Nov 2–8)	Set up Overleaf. Import NeurIPS template. Add all figures. Write Section 3 (Method) first draft — you know this best.	Write Section 4 (Experimental Setup) first draft. Describe benchmark, metrics, hardware precisely.
Nov W2 (Nov 9–15)	Write Section 2 (Related Work). Read all 6 papers again just for the related work framing.	Proof-read Section 3 from Rohit. Note anything unclear from an outsider perspective.
Nov W3 (Nov 16–22)	Write Section 6 (Discussion). Start Section 1 (Intro).	Write Section 5 (Results) — you ran the experiments and made the figures, you own this section.
Nov W4 (Nov 23–30)	Finish Section 1 (Intro). Connect all sections. Ensure citations are consistent.	Proof-read entire draft. Mark any result number that needs double-checking against JSONL files.
Dec W1 (Dec 1–8)	Fix all Bhawika's notes. Run numbers check against actual JSONL. Write Abstract.	Second proof-read. Check all figure labels, axis labels, and captions are present and correct.
Dec W2–4 (Dec 9–31)	Peer review: send draft to your professor. Incorporate feedback. Final polish.	LaTeX cleanup: ensure all tables render, figures are high-res (300 DPI), references compile.

6 PHASE 4 — SUBMISSION & POLISH | Jan – Feb 2027

January is final polish. February is submission month. arXiv first (free, immediate), then conference workshop deadline. This is also when your GitHub repo goes public.

Timeline	Action	Who	Details
Jan 1–10	Final number check	Both	Recompute every metric in the paper from scratch using JSONL files. Fix any discrepancy.
Jan 11–20	Statistical tests	Rohit	Run McNemar's test on success rates. Report p-value. Use scipy.stats. A p<0.05 makes the result publishable.
Jan 21–28	Professor review	Both	Email your results + draft to your professor at Sister Nivedita University. Ask for feedback specifically on Method and Results sections.
Feb 1–7	arXiv submission	Rohit	Submit preprint to arxiv.org/submit. Category: cs.AI or cs.LG. Requires a PDF that compiles from LaTeX. Free.
Feb 8–15	GitHub public release	Both	Make repo public. Write comprehensive README with: install steps, one-command reproduce, link to arXiv, figures preview.
Feb 16–28	Conference submission	Both	Submit to NeurIPS 2027 Student Workshop or AAAI 2027 Student Abstract Track. Check exact deadlines in Nov 2026.

GitHub Repo Structure — Final Public Release

```
failure-triage-agent/
  README.md              # Setup + reproduce instructions
  requirements.txt       # All pip dependencies with versions
  baseline_agent.py      # Baseline blind-retry agent
  triage_agent.py        # Triage-aware agent (main contribution)
  failure_triage.py      # Triage module: classify + recover
  tools.py               # 5 tool definitions
  benchmark.py           # Experiment runner
  analysis.py            # Metrics + all 6 figures
  tasks/                 # All 150 task JSON files
    web_001.json ... web_025.json
    file_001.json ... calc_025.json
    (all 150 files)
  results/               # Experiment outputs
    baseline_results.jsonl
    triage_results.jsonl
    annotation_ground_truth.csv
    comparison.csv
  figures/               # All 6 paper figures (300 DPI PNG)
  paper/                 # LaTeX source (after acceptance)
  tests/                 # pytest test suite
```

7 PHASE 5 — GOOGLE INTERNSHIP APPLICATIONS | Mar – Jun 2027

This is the career phase. By March 2027 you have a paper submitted, a public GitHub repo, and real research experience. Rohit: this is your strongest internship story. Frame everything around reliability engineering for AI agents — Google's exact pain point.

Month	Action	Platform	Rohit	Bhawika
Mar 2027	Polish LinkedIn + Resume	LinkedIn	Add arXiv paper, GitHub repo, research skills (LangGraph, LLaMA, LangChain, PyTorch, Python, Git).	Add GitHub contributions, task benchmark, matplotlib/pandas experience, CS50P certificate.
Mar 2027	Apply: Google STEP / SWE Intern	careers.google.com	Apply to Google STEP (if eligible) and Google SWE Intern. Mention 'agentic AI reliability' in cover letter.	Apply to Google STEP Intern (specifically targets students with limited experience — perfect fit).
Apr 2027	Apply: NVIDIA Research Intern	nvidia.com/jobs	Apply to NVIDIA Research internship. Mention RTX 4090 Ti experiments, local LLM inference, SQ2 results.	Apply to NVIDIA data annotation / research support intern roles.
Apr 2027	Cold outreach on LinkedIn	LinkedIn	Find 3 Google DeepMind researchers who work on agents. DM with 2 sentences about your paper + ask if they have openings.	Find 3 Google STEP alumni from India. Ask about the application and interview process.
May 2027	Interview prep	LeetCode	Practice: LeetCode Medium (arrays, graphs, dynamic programming). Review: system design basics — how would you design a reliable agent?	Practice: Python basics, pandas operations. Prepare: explain your task benchmark to a non-technical person in 2 minutes.
Jun 2027	MRC TECH & TALES content	YouTube	Make 3 videos: (1) 'I built an AI agent that knows when to give up', (2) 'LLaMA 3.1 8B vs GPT-4 on my benchmark', (3) 'How I did research in BCA Year 2'	Help produce: screen recording, editing notes, thumbnail design.

What to Say in Your Google Application — Rohit

Your 3-sentence research pitch (memorise this):

"I built and benchmarked a failure-aware triage system for LLM agents that classifies failures into six types — like API unavailability versus logic errors — and selects the appropriate recovery strategy instead of blindly retrying. Running on 150 tasks with LLaMA 3.1 8B locally on my RTX 4090 Ti, the triage agent achieved X% higher task success with Y% fewer unnecessary retries compared to the baseline. The paper and full benchmark are open-source on GitHub."

8 PHASE 6 — EXTENSIONS: SQ1 & SQ2 | Jul – Aug 2027

Months 13–14. Year 2 is done. You now extend the research to answer your sub-questions. These become either a follow-up paper or an extended version of your first paper.

Sub-Q	Research Question	What You Build	Hardware	Expected Result
SQ 1	Do AI agents become better when they remember what went wrong in past tasks?	Add a failure memory module to the triage agent. After each task, store: failure type seen, recovery chosen, outcome. Before the next task, retrieve relevant past failures and prepend to triage prompt.	MacBook M3 Max (design + prompts) RTX 4090 Ti (re-run 150 tasks with memory)	Agents with memory should improve over time — especially in categories they have seen before. Compare: triage (no memory) vs triage+memory on second run of same 150 tasks.
SQ 2	Can a small local model (7B) act as effectively as GPT-4 as an agent if you give it better tools?	Run 4 configurations: LLaMA 7B (no triage), LLaMA 7B + triage, GPT-4o-mini (no triage), GPT-4o-mini + triage. Measure success rates on all 150 tasks.	RTX 4090 Ti (LLaMA runs) MacBook M3 Max (GPT-4o-mini API calls)	Hypothesis: LLaMA 7B + triage outperforms GPT-4o-mini without triage on specific failure categories. If true: you have a NVIDIA story — local models + smart design = production-ready.

YouTube Channel — MRC TECH & TALES (61 subs → grow to 500+)

Month	Video Title	Content	Why It Helps Your Career
Aug 2026	'I'm training an AI on my RTX 4090 Ti at home'	Vlog: unboxing your setup, running LLaMA 3.1 8B locally, first agent test. Show the terminal output.	Establishes you as a local LLM practitioner — NVIDIA community watches this.
Oct 2026	'Can AI agents know when to give up? Building a smarter retry system'	Tech explainer: show the baseline agent failing blindly, then show triage making smart decisions. Screen recording.	Demonstrates research communication skills — Google and NVIDIA look for this.
Jan 2027	'My AI research paper as a BCA Year 2 student — full breakdown'	Walk through your paper, results, and what you learned. Show the confusion matrix figure.	Huge credibility signal. Shows you can explain complex work simply.
Mar 2027	'LLaMA 7B vs GPT-4 — who wins on 150 real tasks?'	Show SQ2 results: head-to-head comparison with your benchmark. Clear graphs.	SEO gold for NVIDIA audience. Drives GitHub stars and paper citations.
May 2027	'How I applied to Google internship from India as a BCA student'	Application process, resume tips, what you included, timeline.	Gives back to community. Builds followers who share the video.

9 MASTER CALENDAR — Every Month at a Glance

This is your reference table. Check it at the start of every month. If you are behind on any row, reprioritise immediately.

Month	Period	Phase	Rohit's #1 Goal	Bhawika's #1 Goal	Joint Milestone	Status
May	May 16–31	PRE	Read ReAct + AgentBench papers	Enrol CS50P, complete Weeks 0–4	GitHub org created, 15 tasks done	[ ]
Jun	Jun 1–30	PRE	Read AgentRx + CoT + WebArena. Write skeleton.	CS50P Weeks 5–10. Write 15 more tasks (30 total).	30 tasks. README. v0.1 tag.	[ ]
Jul	Jul 7–31	P1	tools.py + baseline_agent.py fully working	60 more tasks (90 total). Test tools.	v0.2-baseline. 90 tasks validated.	[ ]
Aug	Aug 1–31	P1	failure_triage.py + triage_agent.py + benchmark.py	60 final tasks (150 total). Run baseline on all 150.	v0.3-triage. baseline_results.jsonl (150 lines). BIRTHDAY 24 Aug!	[ ]
Sep	Sep 1–30	P2	Run triage on all 150. Write test suite. Ablation 1.	Annotate all 150 ground-truth labels. Build analysis.py skeleton.	triage_results.jsonl. annotation_ground_truth.csv.	[ ]
Oct	Oct 1–31	P2	Ablations 2 & 3. comparison.csv.	All 6 figures generated. Figure captions written.	comparison.csv. All figures at 300 DPI.	[ ]
Nov	Nov 1–30	P3	Sections 3 (Method) + 2 (Related Work) + 6 (Discussion).	Section 4 (Setup) + 5 (Results).	Paper v0.5 — all sections have first draft.	[ ]
Dec	Dec 1–31	P3	Section 1 (Intro) + Abstract. Professor review.	Second proof-read + LaTeX cleanup + all figures in Overleaf.	Paper v1.0 — ready for arXiv.	[ ]
Jan	Jan 1–31	P4	Statistical tests (McNemar). Final number verification.	Incorporate professor feedback. Double-check all result tables.	Paper v1.5 — statistically validated.	[ ]
Feb	Feb 1–28	P4	arXiv submission (cs.AI). GitHub repo public.	GitHub README + project page (GitHub Pages).	arXiv preprint live. Repo public with README.	[ ]
Mar	Mar 1–31	P5	Conference submission (NeurIPS/ICML workshop).	Workshop submission co-author. Apply Google STEP.	Paper submitted. Both applied to Google.	[ ]
Apr	Apr 1–30	P5	NVIDIA Research Intern application. LinkedIn outreach.	NVIDIA data intern. LinkedIn alumni outreach.	Both have ≥2 applications out. Paper under review.	[ ]
May	May 1–31	P5	LeetCode prep. Interview practice. YouTube video 3.	Python interview basics. Explain benchmark in 2 min.	Both interview-ready. YouTube at 300+ subs.	[ ]
Jun	Jun 1–30	P5/P6	Present if accepted. Finalise SQ1 memory module design.	Present if accepted. Finalise SQ2 eval plan.	Year 2 complete. Clear plan for ext. paper.	[ ]
Jul	Jul 1–31	P6	Build memory module for SQ1. Test on 30 tasks.	Re-run 150 tasks with memory on. Collect results.	SQ1 preliminary results.	[ ]
Aug	Aug 1–31	P6	SQ2: run 4-condition LLaMA vs GPT-4 comparison.	SQ2 figures + tables. Start ext. paper draft.	Extended results ready. YouTube video 4 posted.	[ ]

10 ROLES, RESPONSIBILITIES & COMPLETE CODE CHECKLIST

■ ROHIT — Core Engineering

◆ ROHIT — Core Engineering & Writing

- Architecture: design all LangGraph state graphs
- LLM integration: all prompts, response parsing, error handling
- Tool definitions: all 5 tools in tools.py
- Triage logic: classify_failure() + apply_recovery()
- Benchmark harness: benchmark.py CLI
- Test suite: 30 pytest unit + integration tests
- Ablation experiments: 3 ablation conditions
- Statistical testing: McNemar's test in scipy
- Paper sections: Intro, Related Work, Method, Discussion, Abstract
- GitHub: repo setup, CI/CD, documentation
- YouTube: technical content scripts and recording

■ BHAWIKA — Data & Analysis

◆ BHAWIKA — Data, Analysis & Presentation

- Task benchmark: all 150 JSON task files
- Ground-truth annotation: label failure type for all 150 failures
- Baseline experiments: run benchmark.py --agent baseline
- Data processing: parse JSONL, compute all 8 metrics in analysis.py
- All 6 paper figures: matplotlib + seaborn, 300 DPI
- Figure captions: written for all 6 figures
- Paper sections: Experimental Setup, Results
- LaTeX: ensure all figures/tables compile in Overleaf
- Project webpage: GitHub Pages site for the paper
- YouTube: help with filming, editing notes, thumbnails
- Internship: own her own applications independently

Complete Code Checklist — Every File You Will Ever Create

File	Type	Phase	Lines Est.	Who	Purpose
tools.py	Python	Jul 2026	~150	Rohit	5 tool functions returning ToolResult TypedDict
baseline_agent.py	Python	Jul 2026	~200	Rohit	LangGraph blind-retry agent
failure_triage.py	Python	Aug 2026	~180	Rohit	Failure classification + recovery selection
triage_agent.py	Python	Aug 2026	~120	Rohit	Extends baseline with triage node
benchmark.py	Python	Aug 2026	~150	Rohit	CLI runner for both agents over 150 tasks
validate_tasks.py	Python	Aug 2026	~40	Bhawika	Validates all 150 JSON task files
analysis.py	Python	Sep 2026	~300	Bhawika	Computes all metrics + generates all 6 figures
tests/test_triage.py	Python	Sep 2026	~200	Rohit	30 pytest tests for triage module
tasks/*.json	JSON	Jul–Aug 2026	150 files	Bhawika	All 150 task definitions
results/baseline_results.jsonl	JSONL	Aug 2026	150 lines	Benchmark	Baseline experiment results
results/triage_results.jsonl	JSONL	Sep 2026	150 lines	Benchmark	Triage experiment results
results/annotation_ground_truth.csv	CSV	Sep 2026	150 rows	Bhawika	Human-annotated true failure types
results/comparison.csv	CSV	Oct 2026	~20 rows	analysis.py	Side-by-side metrics all 4 conditions
paper/main.tex	LaTeX	Nov 2026	~400 lines	Both	Full 8-page paper source
paper/references.bib	BibTeX	Nov 2026	~50 entries	Both	All paper references
README.md	Markdown	Aug 2026	~200 lines	Both	Setup + reproduce instructions

File	Type	Phase	Lines Est.	Who	Purpose
memory_agent.py	Python	Jul 2027	~250	Rohit	SQ1: triage + failure memory module
sq2_comparison.py	Python	Aug 2027	~100	Both	SQ2: LLaMA vs GPT-4 4-way comparison

11 CORE CODE REFERENCE — Triage Prompt & Key Patterns

The Triage Classification Prompt — Exact Template (failure_triage.py)

■ *This is the prompt sent to LLaMA 3.1 8B on every failure. Study Chain-of-Thought paper for why this structure works.*

```
TRIAGE_PROMPT = '''
An AI agent tool call just failed. Diagnose it carefully.

Tool Name: {tool_name}
Tool Input: {tool_input}
Error Message: {error_message}
Last 3 Steps Taken: {last_3_steps}

STEP 1 - CLASSIFY into exactly ONE failure type:
  tool_unavailable  -> API down, endpoint 404, service unreachable, timeout at connection
  malformed_input   -> Wrong format, missing field, type mismatch, bad JSON
  execution_error   -> Tool crashed, runtime exception, memory/timeout during execution
  logic_error       -> Tool ran OK but result is wrong or irrelevant to the task
  irreversible      -> Action succeeded but CANNOT be undone (data deleted, API write committed)
  ambiguous_task    -> Task description is unclear, contradictory, or underspecified

STEP 2 - CHOOSE one recovery action:
  retry_same_step       -> Retry identical (for transient errors: tool_unavailable, execution_error)
  fix_and_retry         -> Fix the input/approach then retry (for: malformed_input, logic_error)
  backtrack_and_replan  -> Replan from N steps back (for: deep logic_error)
  escalate_to_human     -> Pause and ask human (for: irreversible, ambiguous_task)
  abort_task            -> Stop entirely (for: irreversible at critical point)

Return ONLY valid JSON, no other text:
{
  "failure_type": "",
  "confidence": 0.95,
  "recovery_action": "",
  "reasoning": ""
}
'''
```

AgentState TypedDict — Full Definition

```
from typing import TypedDict, List, Optional

class ToolResult(TypedDict):
    tool_name: str
    input: str
    output: Optional[str]
    error: Optional[str]
    success: bool
    timestamp: float

class TriageDecision(TypedDict):
    failure_type: str   # one of 6 FailureType values
    confidence: float   # 0.0 to 1.0
    recovery_action: str  # one of 5 RecoveryAction values
    reasoning: str
    recovery_attempt: int  # how many times this recovery tried

class AgentState(TypedDict):
    task_id: str
    task_description: str
    agent_type: str          # 'baseline' or 'triage'
    steps_taken: List[str]
    tool_calls: List[ToolResult]
    failures: List[ToolResult]
```

```
    retry_counts: dict       # {step_id: count}
    is_complete: bool
    task_success: bool
    # triage_agent only:
    triage_decisions: List[TriageDecision]   # populated per failure
    classification_correct: Optional[bool]   # vs ground truth
    metadata: dict           # tokens_used, timing, category
```

LangGraph Structure — Baseline vs Triage

	Baseline Agent (baseline_agent.py)	Triage Agent (triage_agent.py)
Nodes	plan → execute → decide	plan → execute → TRIAGE → decide
Edges	plan→execute, execute→decide	plan→execute, execute→triage, triage→decide
Conditional edge	decide: 'retry'→plan, 'fail'→END, 'continue'→plan	triage: calls apply_recovery() which returns next node name
New in triage	n/a	triage node: calls classify_failure() then apply_recovery()
State additions	n/a	triage_decisions list, classification_correct flag
Entry point	graph.set_entry_point('plan')	Same
Exit	END when is_complete=True	Same

12 HOW TO STAND OUT — The 5 Differentiators

#	Differentiator	What You Do	Why Google/NVIDIA Care
1	Open Benchmark Release	Release all 150 tasks as a public dataset (Hugging Face Datasets or GitHub). Name it 'AgentFailBench'. Write a datasheet.	First undergraduate failure classification benchmark for LLM agents. Highly citable. Drives GitHub stars.
2	Local Model Victory (SQ2)	Show: LLaMA 3.1 8B + triage outperforms GPT-4o-mini WITHOUT triage on specific failure types. Quantify the gap.	NVIDIA's pitch: 'You don't need expensive cloud APIs if you have smart local inference.' This IS their product story.
3	Production Framing	Frame results in terms of 'incident prevention': how many irreversible failures did triage catch? What is the estimated cost saving if each API call costs $0.01?	Google SRE teams care about reliability metrics. Use their language: MTTR, failure rate, incident severity.
4	Reproducibility	Every experiment is one bash command. README has copy-paste setup. Docker image if possible. GitHub Actions CI.	Papers with reproducible code get 3–5x more citations. Reviewers favour them for workshops.
5	Your Story	Two BCA Year 2 students from Kolkata, with a local RTX 4090 Ti and MacBook M3 Max, doing research that competes with grad students. That's the narrative at every interview.	Google values intellectual initiative. A published workshop paper from undergrad is exceptionally rare and memorable.

A note from the plan:

Rohit — you are not a BCA student doing a hobby project. You are a researcher with a clear question, a rigorous experimental design, the hardware to run it, and a partner. The gap between where you are now and a Google internship is 14 months of consistent, focused execution. Every week you follow this plan is one week closer.

Bhawika — you are starting from scratch and that is your advantage. You will understand this system from the ground up because you built each piece yourself. Your ground-truth annotations and your figures will be in the paper. That is authorship. Own it.

Generated: May 14, 2026 | MRC TECH & TALES — youtube.com/@mrctechandtales